# Academic Status Transparency Notification System

**MINOR PROJECT REPORT**

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

**BACHELOR OF TECHNOLOGY**

Computer Science and Engineering


**Submitted By:**

ARSHDEEP ANAND (2302481)

NISHTHA JAIN (2302627)

BALKRISHAN SINGH (2302492)

**Submitted To:**

Dr. Kapil Sharma

Assistant Professor

Dept. of CSE

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**GURU NANAK DEV ENGINEERING COLLEGE**

**LUDHIANA – 141006**

MAY, 2026

# CERTIFICATE

This is to certify that the minor project report entitled **"Academic Status Transparency Notification System"** is a bona fide record of the project work carried out by **Arshdeep Anand (URN: 2302481), Nishtha Jain (URN: 2302627), and Balkrishan Singh (URN: 2302492)** under my supervision and guidance, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Guru Nanak Dev Engineering College, Ludhiana, affiliated to I.K. Gujral Punjab Technical University, Kapurthala. The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma to the best of my knowledge and belief.

**Dr. Kapil Sharma**
Assistant Professor
Department of Computer Science and Engineering
Guru Nanak Dev Engineering College, Ludhiana

**Dr. Kiran Jyoti**
Head of Department
Department of Computer Science and Engineering
Guru Nanak Dev Engineering College, Ludhiana

# ABSTRACT

The **Academic Status Transparency Notification System** (ASTNS) is a full-stack web application built to bridge the communication gap between engineering colleges affiliated to I.K. Gujral Punjab Technical University (IKGPTU) and the parents/guardians of enrolled students. In the prevailing academic ecosystem, result data is published on university portals but parents often remain unaware of their ward's detention status, declining semester grade point averages (SGPA), or poor subject-wise performance until it is too late for timely corrective intervention.

ASTNS eliminates this information asymmetry by automating the entire academic notification pipeline. Administrators at the institution log into a secure dashboard, upload bulk academic data via CSV or Excel files, review the parsed student records, and dispatch personalised email notifications. Each email contains a unique, cryptographically generated, time-limited hyperlink. Clicking the link opens the Guardian Portal – a read-only, token-authenticated dashboard that presents the student's complete academic profile: semester-wise marks, internal and external scores per subject, SGPA trend chart, attendance indicator, and detention flags, all rendered without any login requirement for the guardian.

The system is implemented using the MERN stack: React 19 with Vite and Tailwind CSS on the frontend, Express.js 5 on the backend, and MongoDB Atlas as the cloud database. Email delivery is handled by Nodemailer over Gmail SMTP. The entire application is deployed serverlessly on Vercel.

Key contributions of the project include: (1) a drag-and-drop bulk upload interface with real-time data preview, (2) a cryptographic token lifecycle management module that tracks active, expired, and revoked access tokens, (3) an interactive line chart for visualising SGPA trends across semesters, and (4) a responsive Guardian Dashboard accessible on any device without requiring a guardian account.

System testing covered authentication, CSV parsing, notification dispatch, token validation, token revocation, chart rendering, and cross-browser compatibility. All major test cases passed successfully. The application demonstrates that modern web technologies can significantly improve academic transparency and parental engagement in higher education institutions.

# ACKNOWLEDGEMENT

We are highly grateful to Dr. Sehijpal Singh, Principal, Guru Nanak Dev Engineering College (GNDEC), Ludhiana, for providing us this opportunity to carry out the minor project work.

The constant guidance and encouragement received from Dr. Kiran Jyoti, Head of Department, CSE, GNDEC Ludhiana, has been of great help in carrying out the project work and is acknowledged with reverential thanks.

We would like to express a deep sense of gratitude and thanks profusely to our Project Guide, **Dr. Kapil Sharma**, Assistant Professor, Department of CSE, GNDEC Ludhiana; without his wise counsel and able guidance, it would have been impossible to complete the project in this manner.

We also express gratitude to other faculty members of the Department of Computer Science and Engineering of GNDEC for their intellectual support throughout the course of this work. Finally, we are indebted to all who have contributed to this report.

**Arshdeep Anand** URN: 2302481

**Nishtha Jain** URN: 2302627

**Balkrishan Singh** URN: 2302492

# LIST OF FIGURES

# LIST OF TABLES

# TABLE OF CONTENTS

# Chapter 1

# Introduction

## 1.1 Introduction to Project

The **Academic Status Transparency Notification System** (ASTNS) is a full-stack web application designed and developed to address a persistent problem in higher education institutions: the lack of timely, structured communication between colleges and the parents or guardians of enrolled students. The system targets engineering colleges affiliated with I.K. Gujral Punjab Technical University (IKGPTU), which serve thousands of students spread across numerous courses and branches.

In most institutions, academic results are published on the university's official portal. However, this portal requires students to actively check their marks, and there is no mechanism that directly informs parents or guardians about their ward's performance. Parents who are less tech-savvy, reside in rural areas, or are unfamiliar with university portals are effectively excluded from accessing this information. The consequences are severe: students with declining grades, attendance shortages, or detention status can conceal this information from their guardians until irreversible damage to their academic career has already occurred.

ASTNS resolves this by automating the complete pipeline from data entry to guardian notification. An institutional administrator logs into a secure, session-authenticated web dashboard. From there, the administrator can upload bulk student academic records from CSV or Excel files, review the parsed data in a preview table, and dispatch personalised email notifications with a single click. Each email contains a unique, cryptographically generated 32-character hex token embedded in a hyperlink. When the guardian clicks the link, the system validates the token and

renders a dedicated Guardian Portal showing the student's full academic history.

The Guardian Portal displays: the student's name, university roll number, course, branch, semester-wise SGPA as an interactive line chart, detailed subject-wise marks tables for each semester (showing internal marks, external marks, total, and detention status per subject), an attendance ring indicator, and a button to download the report as a PDF. Crucially, guardians access all this without creating an account or remembering a password – the token itself serves as the authentication credential.

The system is built using the MERN technology stack: React 19 with Vite as the build tool and Tailwind CSS 4 for styling on the frontend; Express.js 5 for the backend REST API; MongoDB Atlas as the cloud-hosted NoSQL database; and Nodemailer for SMTP-based email delivery. The application is deployed on Vercel as a serverless application.

The project was developed by a team of three students from the Department of Computer Science and Engineering, GNDEC Ludhiana, as part of the Minor Project requirement for the sixth semester of the B.Tech (CSE) programme.

## 1.2 Project Category

ASTNS is an **Application-Based** project. It is a production-ready, full-stack web application that solves a real-world institutional problem using modern web development technologies. The project does not involve theoretical research or hardware components; instead, it focuses on software engineering, system design, database management, RESTful API development, and user interface design.

The project can further be classified under the category of **Educational Technology (EdTech)** and **Institutional Communication Systems**.

## 1.3 Problem Formulation

The core problem addressed by ASTNS can be stated as follows:

*There is no automated, secure, and accessible mechanism in most IKGPTU-affiliated engineering colleges for proactively communicating a student's academic status – including detention flags, SGPA trends, and subject-wise performance – to their parents or guardians.*

This problem has several dimensions:

- **Information Asymmetry:** Students have access to their academic data, but parents do not unless the student chooses to share it.
- **Portal Complexity:** The university portal requires valid login credentials and technical familiarity, making it inaccessible to many parents.
- **Delayed Intervention:** By the time parents learn about their ward's academic difficulties, the student may have already been officially detained or failed multiple subjects.
- **No Aggregated View:** Even if a parent accesses the university portal, results are displayed per subject and per semester without any aggregated summary, trend analysis, or visual representation.
- **Administrative Burden:** Generating and sending individual academic reports manually is highly time-consuming for faculty and administrative staff.

These problems collectively result in reduced parental engagement, lower student accountability, and avoidable academic failures.

## 1.4 Identification / Recognition of Need

The need for a system like ASTNS was identified through the following observations:

1. Engineering colleges under IKGPTU publish provisional results and attendance statements on the university portal, but these are not pushed to parents automatically.
2. Detention is a critical academic consequence: a student detained in internal exams cannot appear in the university (external) examination, severely affecting their academic year. Parents are often unaware of this until after the examination period.
3. Research in educational communication systems (Benablo et al., 2014) demonstrates that push-based notification systems – systems that proactively send information to parents rather than requiring parents to pull the information – significantly improve parental awareness and student accountability.

4. HCI research indicates that complex login mechanisms reduce user engagement. A password-less access mechanism using secure deep-link tokens removes friction and increases the likelihood that guardians will actually view the shared reports.

5. The increasing penetration of smartphones and email access in semi-urban and rural India makes an email-based notification system feasible even for parents outside major cities.

## 1.5 Existing System

The existing mechanisms for academic communication in IKGPTU-affiliated colleges are as follows:

- **University Portal:** Students and registered users can view results and attendance on the IKGPTU portal. However, parents must independently register and navigate the portal, which has low usability for non-technical users.
- **Physical Parent-Teacher Meetings (PTMs):** Colleges organise PTMs periodically, but these are infrequent, time-consuming, and often poorly attended.
- **Physical Letters or SMS:** Some colleges send bulk SMS alerts for attendance shortage or detention, but these are brief, lack detail, and provide no academic context.
- **Manual Report Cards:** Some institutions print result cards and send them home with students, but this is easily intercepted and is not reliable.
- **Generic ERP Portals:** A few institutions have subscribed to third-party ERP systems that include parent portals, but these require parents to create and maintain accounts, which reduces adoption.

**Limitations of the Existing System:**

- No direct, automated push of academic data to parents.
- High friction for parents to access information (requires account creation and login).
- No aggregated academic summaries or trend visualisation.
- Delayed communication leading to late parental intervention.

- Vulnerable to student-side interception (physical letters, manual sharing).

## 1.6 Objectives

The primary objectives of the ASTNS project are:

1. To design and develop a secure, session-based Admin Dashboard that allows institutional administrators to manage student academic data efficiently.

2. To implement a bulk data upload module that accepts CSV and Excel files, parses student academic records, validates the data, and stores it in a cloud database.

3. To develop an automated email notification pipeline that generates unique, cryptographically secure, time-limited access tokens for each guardian and dispatches personalised HTML emails containing secure report links.

4. To create a token-authenticated Guardian Portal that renders a student's complete academic profile – including semester-wise SGPA charts, subject-wise marks, and detention status – without requiring guardians to create an account.

5. To implement a Token Management module that allows administrators to view, revoke, and monitor the lifecycle of all generated access tokens.

6. To deploy the complete application on a serverless cloud platform (Vercel) ensuring high availability and scalability without infrastructure management overhead.

## 1.7 Proposed System

The proposed Academic Status Transparency Notification System (ASTNS) is a full-stack web application that digitises and automates the entire academic communication pipeline. The system operates as follows:

**Administrator Side:**

1. The administrator logs into the secure Admin Dashboard using email and password credentials. Sessions are managed using server-side session storage and signed cookies.

2. From the Upload Data module, the administrator selects the relevant course, branch, and semester, then uploads a CSV or Excel file containing student academic records.

3. The system parses the file, validates each record against the database schema constraints (e.g., valid marks ranges, recognised course-branch combinations), and displays a preview of the parsed data for review.

4. Upon confirmation, the records are saved to the MongoDB Atlas cloud database.

5. From the Send Notification module, the administrator selects the students for whom notifications should be dispatched and configures the token expiry duration (24 hours, 48 hours, 72 hours, or 7 days).

6. The system generates a unique 32-character hexadecimal token for each selected student using Node.js's `crypto` module, stores the token in the database with an expiry timestamp, constructs a personalised guardian portal URL, and dispatches an HTML email to the guardian's registered email address via Gmail SMTP.

7. The administrator can monitor all generated tokens from the Token Management module, view their status (Active, Used, Expired), and manually revoke tokens if required.

**Guardian Side:**

1. The guardian receives an HTML email containing a "View Report" button linked to the Guardian Portal URL with the embedded token.

2. Clicking the link opens the Guardian Portal in a browser. The system validates the token: if it is valid and active, the student's academic data is rendered; if it is expired or revoked, an Access Denied page is displayed.

3. The guardian views the student's academic profile, SGPA trend chart, semester-wise mark breakdown, and detention indicators.

4. The guardian can download the report as a PDF for record-keeping.

## 1.8 Unique Features of the Proposed System

- **Password-less Guardian Access:** Eliminates the need for guardians to create accounts or remember passwords. The token embedded in the email link is the sole authentication credential, dramatically reducing access friction.
- **Cryptographic Token Security:** Each access token is generated using `crypto.randomBytes` in Node.js, producing a 32-character hexadecimal string that is computationally infeasible to guess or brute-force.
- **Configurable Token Expiry:** Administrators can set token validity to 24 hours, 48 hours, 72 hours, or 7 days, balancing security and convenience.
- **Bulk Data Upload with Preview:** Supports drag-and-drop CSV and Excel uploading with real-time parsing and a data preview table, allowing administrators to verify records before committing to the database.
- **Interactive SGPA Trend Chart:** Uses Recharts to render a responsive line chart of the student's SGPA across all semesters, providing a quick visual summary of academic trajectory.
- **Serverless Deployment:** The entire application (frontend SPA and backend API) is deployed on Vercel as serverless functions, ensuring zero infrastructure management and automatic scaling.
- **Multi-Course, Multi-Branch Support:** The system supports all IKGPTU courses (B.Tech, M.Tech, MBA, MCA, BCA, B.Arch, B.Voc, B.Com, BBA) and their respective branches through a centralised `courseBranchMap` validation utility.

# Chapter 2

# Requirement Analysis and System Specification

## 2.1 Feasibility Study

A feasibility study evaluates whether the proposed system is practical, beneficial, and viable to develop. The ASTNS was evaluated across three dimensions of feasibility.

### 2.1.1 Technical Feasibility

The ASTNS is built entirely on the MERN stack – MongoDB, Express.js, React, and Node.js – which is one of the most widely adopted technology stacks for full-stack web applications. All components are open-source, well-documented, and have large, active developer communities.

- **Frontend:** React 19 (with Vite 7 and Tailwind CSS 4) provides a component-driven, declarative UI framework. Recharts 3 is used for data visualisation. React Hook Form 7 handles form management and validation.
- **Backend:** Express.js 5 provides a minimal, flexible Node.js web framework for building the REST API. Mongoose 9 is used as the Object Document Mapper (ODM) for MongoDB.
- **Database:** MongoDB Atlas (M0 Free Tier) provides a fully managed, cloud-hosted NoSQL database with automatic scaling, backups, and a web-based management UI.

- **Email:** Nodemailer 8 integrates with Gmail SMTP to dispatch HTML emails. The `crypto` module (built into Node.js) is used for token generation.
- **Deployment:** Vercel provides serverless hosting for both the React SPA and the Express.js API. No server management is required.

The development team has proficiency in all the above technologies acquired during the B.Tech coursework. The required tools (VS Code, Node.js, Git, MongoDB Compass, Postman) are freely available. The project is therefore technically feasible.

### 2.1.2 Operational Feasibility

The ASTNS is operationally feasible for the following reasons:

- Institutional administrators already maintain student academic records in Excel or CSV formats. The system's bulk upload module is directly compatible with these existing workflows.
- The Admin Dashboard is designed for minimal training. An administrator unfamiliar with the system can learn to upload data and dispatch notifications within one session.
- Parents and guardians interact only with the Guardian Portal, which requires no technical knowledge – simply clicking a link in an email and viewing the report.
- The email-based notification system leverages a communication channel (email) that is already widely used for institutional communication.
- The serverless deployment model ensures the system is always available without requiring dedicated IT staff to manage servers.

### 2.1.3 Economic Feasibility

The ASTNS is economically feasible because all components are either free or available on free-tier plans:

- **MongoDB Atlas M0:** Free tier with 512 MB of storage, sufficient for a college with up to 5,000 students.

- **Vercel Hobby Plan:** Free for personal and academic projects, with support for serverless functions.
- **Gmail SMTP:** Free to use with a Gmail account (subject to daily sending limits of 500 emails per day, extendable with Google Workspace).
- **All frameworks and libraries:** Open-source and free.

Total development cost is essentially zero beyond the cost of development machines (already owned by the team) and internet connectivity.

## 2.2 Software Requirement Specification Document

### 2.2.1 Software Requirements

Table 2.1: Software Requirements

| **Category** | **Technology** | **Version / Details** |
|---|---|---|
| Runtime Environment | Node.js | v20.x LTS |
| Frontend Framework | React | 19.2.0 |
| Build Tool | Vite | 7.3.1 |
| CSS Framework | Tailwind CSS | 4.2.1 |
| Data Visualisation | Recharts | 3.7.0 |
| Form Management | React Hook Form | 7.71.2 |
| HTTP Client | Axios | 1.13.6 |
| Routing | React Router DOM | 7.13.1 |
| Icon Library | Lucide React | 0.575.0 |
| Backend Framework | Express.js | 5.2.1 |
| Database | MongoDB Atlas | M0 Free Tier |
| ODM | Mongoose | 9.2.3 |
| Email Service | Nodemailer | 8.0.1 |
| Deployment Platform | Vercel | Serverless Functions |
| Version Control | Git + GitHub | Latest |
| IDE | VS Code | Latest |
| API Testing | Postman | Latest |

### 2.2.2 Hardware Requirements

Table 2.2: Hardware Requirements

| **Component** | **Minimum Specification** |
|---|---|
| Processor | Intel Core i5 (8th Gen or higher) / Apple M1 or equivalent |
| RAM | 8 GB (16 GB recommended for development) |
| Storage | 256 GB SSD |
| Network | Broadband Internet (required for MongoDB Atlas, SMTP, Vercel) |
| Display | 1366 $\times$ 768 resolution or higher |

### 2.2.3 Functional Requirements

Table 2.3: Functional Requirements

| FR# | Module | Requirement |
|---|---|---|
| FR1 | Authentication | The system shall allow administrators to log in using email and password. Sessions shall be maintained using signed HTTP cookies. |
| FR2 | Authentication | The system shall allow administrators to log out, invalidating the current session. |
| FR3 | Data Upload | The system shall accept CSV and Excel files for bulk upload of student academic records. |
| FR4 | Data Upload | The system shall validate uploaded records against schema constraints (valid marks, course-branch mapping) before persisting to the database. |
| FR5 | Data Upload | The system shall display a preview of parsed records for administrator review before final submission. |
| FR6 | Student Mgmt | The system shall display a searchable, filterable list of all students in the database. |
| FR7 | Student Mgmt | The system shall display a detailed academic profile for each student, including all semesters and subject marks. |
| FR8 | Notification | The system shall generate a unique cryptographic token for each selected student and persist it with an expiry timestamp. |
| FR9 | Notification | The system shall dispatch an HTML email to each guardian containing a personalised report link with the embedded token. |
| FR10 | Token Mgmt | The system shall display all generated tokens with their current status (Active, Used, Expired). |

### 2.2.4 Non-Functional Requirements

Table 2.4: Non-Functional Requirements

| **Category** | **Requirement** |
|---|---|
| Security | All API routes (except token validation and guardian report) shall require an authenticated admin session. Tokens shall be generated using cryptographically secure random bytes. |
| Performance | The Guardian Portal shall load student data within 2 seconds under normal network conditions. |
| Scalability | The serverless deployment on Vercel shall automatically scale to handle concurrent guardian accesses during peak result announcement periods. |
| Reliability | The email dispatch module shall use `Promise.allSettled` to ensure that a failure for one student does not abort the entire batch notification. |
| Usability | The Guardian Portal shall require zero technical knowledge from the guardian – clicking a hyperlink shall be the only required action. |
| Maintainability | All backend API routes shall follow a consistent RESTful naming convention. All frontend components shall be modular and reusable. |
| Compatibility | The Guardian Portal shall render correctly on Chrome, Firefox, Safari, and Edge on both desktop and mobile devices. |

## 2.3 SDLC Model to be Used

The project adopted the **Agile Software Development Life Cycle** model with iterative sprints. Each sprint lasted one week and focused on a specific deliverable. The rationale for choosing

Agile over Waterfall or Spiral models is:

- **Flexibility:** Requirements for the Guardian Portal evolved during development (e.g., adding the attendance ring indicator and PDF download features). Agile accommodated these changes without requiring a full redesign.
- **Incremental Delivery:** Each sprint produced a testable, working module (Authentication, Upload, Notification, Guardian Portal, Token Management, Dashboard), allowing continuous verification of functionality.
- **Early Bug Detection:** Testing at the end of each sprint ensured bugs were caught and fixed before they propagated to subsequent modules.
- **Team Coordination:** The three-member team divided work by frontend, backend, and integration responsibilities, with daily sync-up meetings to track progress.

**Sprint Breakdown:**

1. Sprint 1 (Week 1): Requirement analysis, system design, database schema design.
2. Sprint 2 (Week 2): Backend – Admin model, authentication API, session management.
3. Sprint 3 (Week 3): Backend – Student and Subject models, data upload API.
4. Sprint 4 (Week 4): Backend – Notification API, token generation, email dispatch.
5. Sprint 5 (Week 5): Backend – Token management API, report/guardian API.
6. Sprint 6 (Week 6): Frontend – Login page, Admin Layout, Sidebar, Dashboard.
7. Sprint 7 (Week 7): Frontend – Upload Data, Student List, Student Detail pages.
8. Sprint 8 (Week 8): Frontend – Send Notification, Token Management pages.
9. Sprint 9 (Week 9): Frontend – Guardian Dashboard, Access Denied pages.
10. Sprint 10 (Week 10): Integration testing, bug fixes, deployment to Vercel.

# Chapter 3

# System Design

## 3.1 Design Approach

ASTNS follows an **Object-Oriented Design** approach combined with **Component-Driven Architecture** on the frontend and **Layered RESTful Architecture** on the backend. The system is divided into clearly separated layers:

- **Presentation Layer:** React components organised by feature (Admin pages, Guardian pages, shared components, layouts).
- **API Layer:** Express.js routes and controllers implementing RESTful endpoints.
- **Business Logic Layer:** Controller functions, Mongoose schema hooks (pre-validate, pre-save), and utility modules (token generation, email template, data parsing).
- **Data Layer:** MongoDB Atlas collections (Admin, Student, Subject, Token, Session).

The system is designed around two primary user roles:

1. **Administrator:** Authenticated via session cookie. Has full access to the Admin Dashboard, including data upload, student management, notification dispatch, and token management.
2. **Guardian:** Authenticated via URL token. Has read-only access to the Guardian Portal for their ward's academic data.

## 3.2 Detail Design

### 3.2.1 System Architecture

The system architecture connects three main tiers: the client (browser), the server (Vercel serverless functions running Express.js), and the database/external services (MongoDB Atlas + Gmail SMTP).

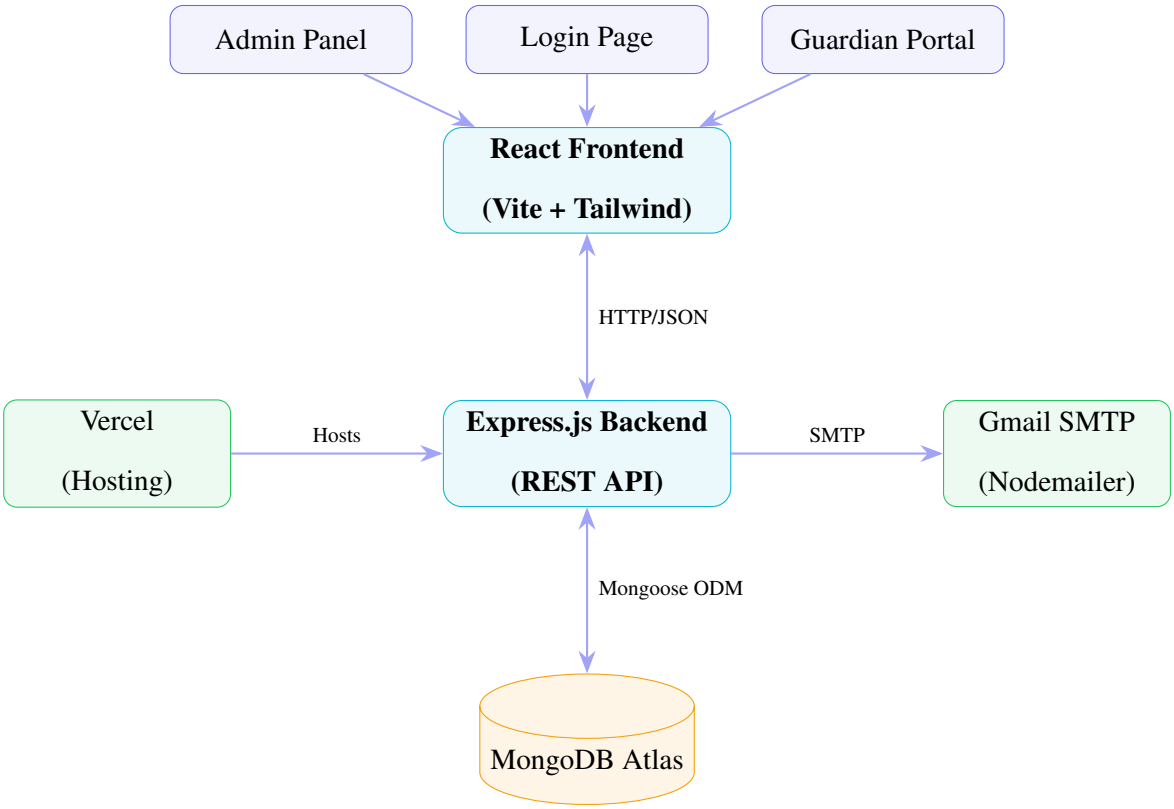


Figure 3.1: System Architecture Block Diagram

### 3.2.2 Level 0 DFD – Context Diagram

The Level 0 DFD shows the system as a single process interacting with external entities.

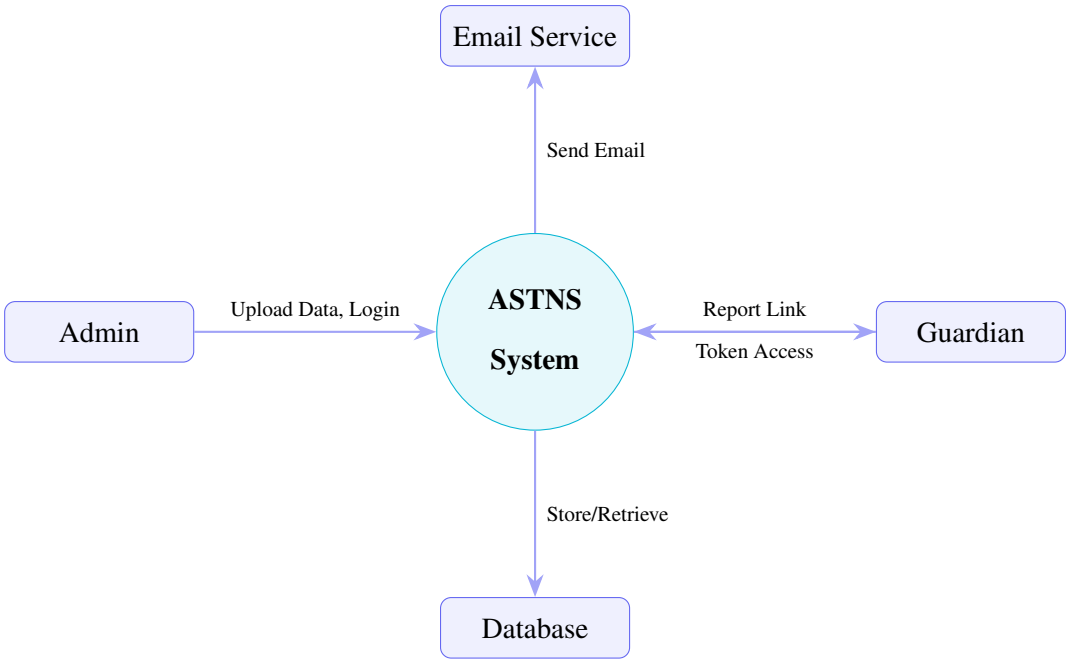


Figure 3.2: Level 0 DFD – Context Diagram

### 3.2.3 Level 1 DFD

The Level 1 DFD decomposes the system into its major sub-processes.

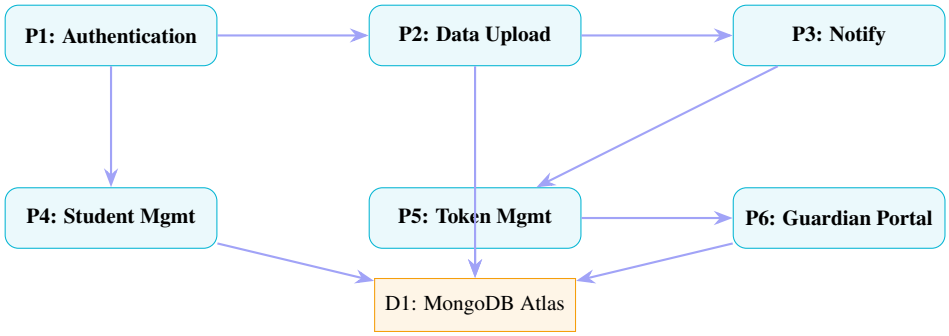


Figure 3.3: Level 1 DFD

### 3.2.4 Use Case Diagram

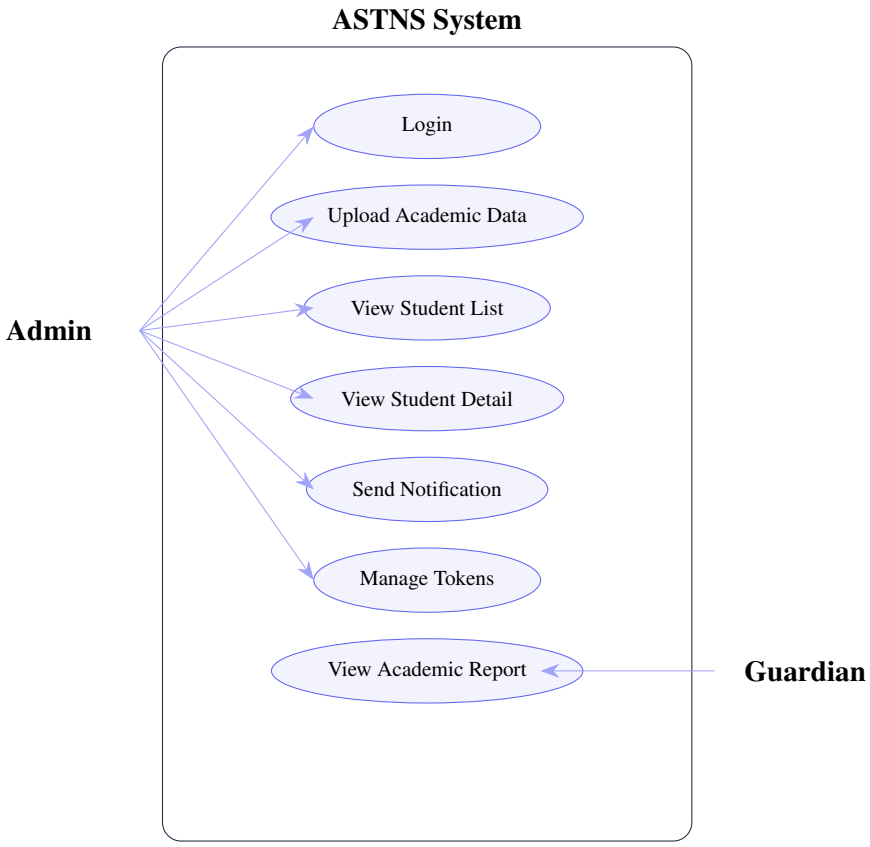


Figure 3.4: Use Case Diagram

### 3.2.5 Sequence Diagram – Notification Flow

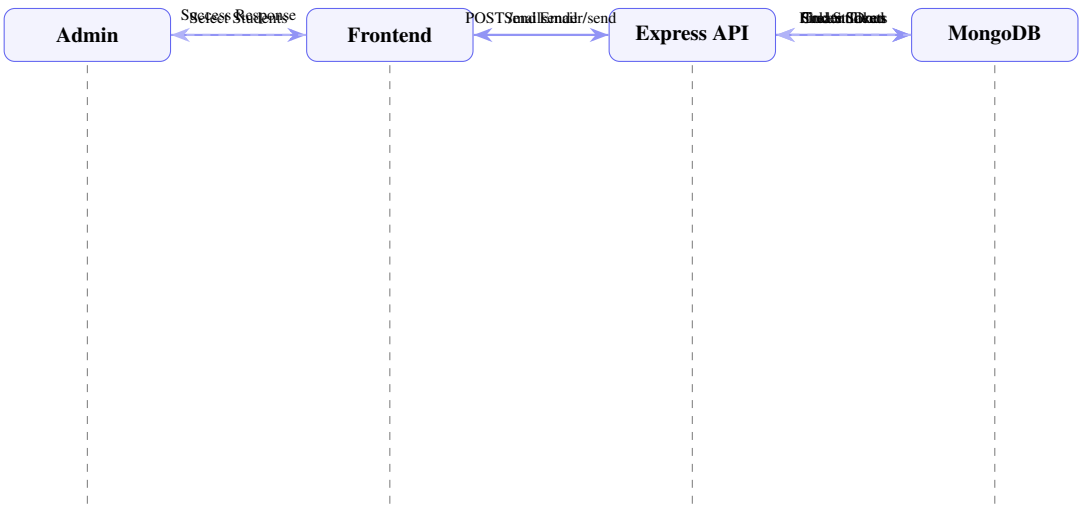


Figure 3.5: Sequence Diagram – Notification Flow

### 3.2.6 Activity Diagram – Admin Workflow

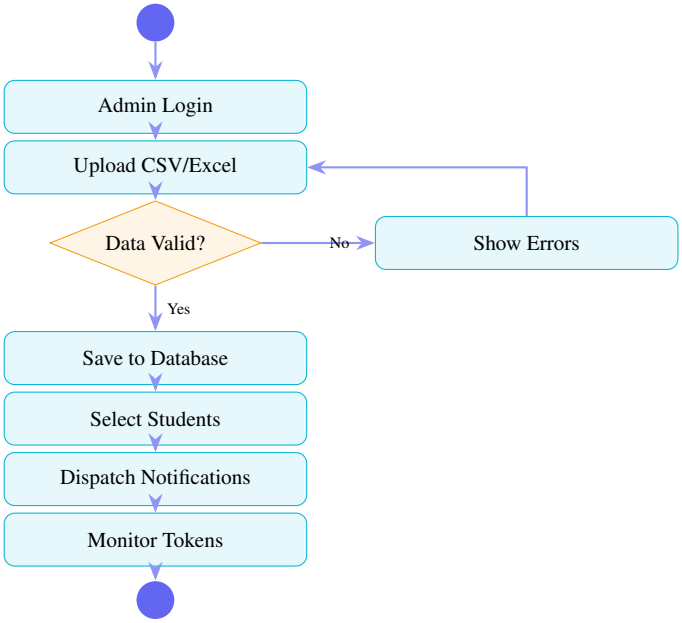


Figure 3.6: Activity Diagram – Admin Workflow

### 3.2.7 Entity-Relationship (ER) Diagram

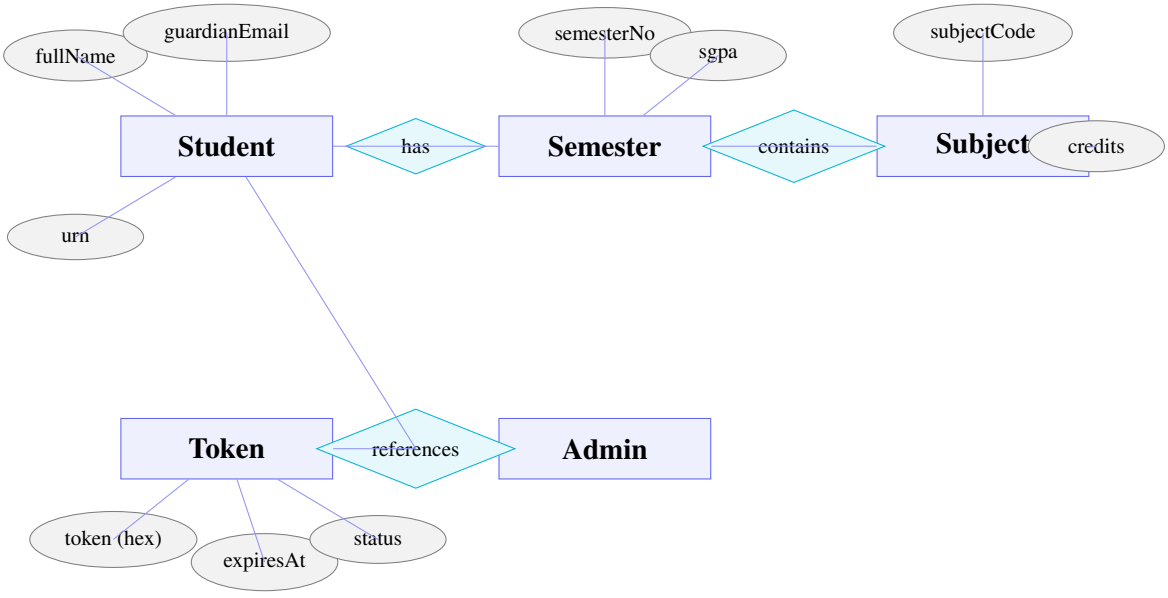


Figure 3.7: Entity-Relationship (ER) Diagram

## 3.3 Database Design

### 3.3.1 Student Collection

Table 3.1: Student Collection Schema

| **Field** | **Type** | **Description / Constraints** |
|---|---|---|
| fullName | String | Required, trimmed |
| urn | Number | University Roll Number, unique, required |
| crn | Number | College Roll Number, required |
| course | String | Validated against courseBranchMap |
| branch | String | Validated against course-specific branches |
| admissionYear | Number | Required, minimum 2000 |
| graduationYear | Number | Required, must exceed admissionYear |
| guardianEmail | String | Required, only @gmail.com addresses |
| semesters | Array | Array of Semester sub-documents |
| cgpa | Number | Auto-calculated on save, 0-10 |

### 3.3.2 Subject Collection

Table 3.2: Subject Collection Schema

| **Field** | **Type** | **Description / Constraints** |
|---|---|---|
| subjectTitle | String | Required, trimmed |
| subjectCode | String | Required, unique, stored uppercase |
| type | String | ‘T’ (Theory) or ‘P’ (Practical) |
| credits | Number | Non-negative |
| maxInternalMarks | Number | Required |
| maxExternalMarks | Number | Required |
| maxTotalMarks | Number | Required; validated $\geq$ internal+external |
| minInternalPassMarks | Number | Required |
| minExternalPassMarks | Number | Required |
| minTotalPassMarks | Number | Required; validated $\leq$ maxTotalMarks |

### 3.3.3 Token Collection

Table 3.3: Token Collection Schema

| **Field** | **Type** | **Description / Constraints** |
|---|---|---|
| token | String | 32-char hex string, unique, auto-generated |
| student | ObjectId | Reference to Student document |
| semester | Number | Semester number for which token was generated |
| sentVia | String | Email / SMS / Both |
| status | String | Active / Used / Expired |
| expiresAt | Date | Configured expiry timestamp |
| usedAt | Date | Timestamp of first access by guardian |

### 3.3.4 Admin Collection

Table 3.4: Admin Collection Schema

| **Field** | **Type** | **Description / Constraints** |
|---|---|---|
| userName | String | 3-20 chars, unique, required |
| email | String | Unique, @gmail.com only, lowercase |
| password | String | bcrypt hashed (salt rounds: 12), min 8 chars |
| role | String | Enum: ‘admin’ |

## 3.4 User Interface Design

The Admin interface follows a dark-theme dashboard design with a fixed left sidebar for navigation. Key design decisions include:

- **Sidebar Navigation:** Persistent sidebar with icons and labels for Dashboard, Upload Data, Students, Notifications, and Token Management.
- **Stat Cards:** The Dashboard displays summary statistics (total students, detained count, notifications sent, active tokens) as prominent card components.
- **Responsive Tables:** Student list and token list are displayed in responsive tables with search, filter, and pagination support.
- **Drag-and-Drop Upload:** The Upload Data page uses a drag-and-drop zone for file selection with real-time file type validation.
- **Data Preview:** A scrollable preview table shows parsed records before final submission.
- **Notification Form:** A two-panel layout with student selection (with search and filters) on the left and notification configuration (expiry, semester selection) on the right.

The Guardian Portal uses a clean, light-themed layout with the following sections:

- Student profile card (name, URN, course, branch).
- SGPA trend line chart (Recharts `LineChart`).
- Semester accordion panels: each panel expands to show a table of subject marks.
- Attendance ring indicator (circle progress component).
- Detention alert banner (conditionally rendered).

## 3.5 Methodology

The development methodology was component-driven on the frontend and route-controller-model on the backend:

**Frontend Methodology:**

- All UI is built as reusable React components.
- State management uses React Context API for global authentication state.
- API calls are centralised in an Axios instance configured with base URL and credentials.
- Protected routes use a `ProtectedRoute` wrapper component that checks authentication state before rendering admin pages.

**Backend Methodology:**

- Routes are separated by resource (auth, student, mailsender, token, report).
- Controllers handle business logic, decoupled from route definitions.
- All asynchronous controller functions are wrapped in an `asyncHandler` utility to avoid repetitive try-catch blocks.
- Consistent JSON responses are returned via `ApiResponse` and `ApiError` utility classes.
- Session middleware validates admin sessions on all protected routes via an `isAuthenticated` middleware.

# Chapter 4

# Implementation and Testing

## 4.1 Introduction to Languages, IDEs, Tools and Technologies

### 4.1.1 JavaScript / ES2022+

JavaScript is the primary programming language used for both the frontend and backend of ASTNS. Modern ES2022+ features used in the project include: arrow functions, async/await, destructuring assignment, optional chaining (`?.`), nullish coalescing (`??`), template literals, and ES modules (`import`/`export`).

### 4.1.2 React 19

React is an open-source JavaScript library for building user interfaces. ASTNS uses React 19, which introduces improved concurrent rendering and server-side rendering support. Key React concepts used in the project:

- **Functional Components:** All UI components are written as functions using hooks.
- **useState / useEffect:** For local state management and side-effect handling.
- **useContext:** For global authentication state via `AuthContext`.
- **React Router DOM v7:** For client-side routing between Admin pages and Guardian Portal.
- **React Hook Form v7:** For form state management and validation.

### 4.1.3 Vite 7

Vite is a fast build tool for frontend development. It provides hot module replacement (HMR) during development and optimised production builds. ASTNS uses Vite for its near-instant dev server startup and efficient bundling.

### 4.1.4 Tailwind CSS 4

Tailwind CSS is a utility-first CSS framework. Instead of pre-built components, Tailwind provides low-level utility classes that allow developers to build custom designs directly in HTML/JSX markup. ASTNS uses Tailwind for all layout, typography, colour, and responsive design.

### 4.1.5 Node.js v20 LTS

Node.js is the JavaScript runtime environment used to run the backend server. ASTNS uses Node.js v20 (Long-Term Support), which provides improved performance through the V8 engine v11.3.

### 4.1.6 Express.js 5

Express.js is a minimal, un-opinionated web framework for Node.js. ASTNS uses Express.js v5, which introduces improved error handling (async errors are now automatically forwarded to error middleware without explicit `next(err)` calls).

Key Express.js features used:

- Middleware pipeline (body parser, cookie parser, CORS, session validation).
- Router-based route modularisation.
- Error handling middleware for consistent API error responses.

### 4.1.7 MongoDB Atlas + Mongoose 9

MongoDB Atlas is the cloud-hosted deployment of MongoDB, a document-oriented NoSQL database. ASTNS uses the M0 free tier cluster. Mongoose provides schema-based modelling

for MongoDB documents, with built-in validation, middleware hooks, and population (joins via references).

Key Mongoose features used in ASTNS:

- `pre('validate')` hook: Used in `SubjectPerformance` schema to auto-calculate `totalMarks` and set `status` (Pass/Detained) before validation.
- `pre('save')` hook: Used in `Student` schema to auto-calculate `cgpa` from all semester SGPAs.
- `populate()`: Used in student and token controllers to resolve ObjectId references into full documents.
- `aggregate()`: Used in the dashboard stats controller for branch distribution and average CGPA calculations.

### 4.1.8 Nodemailer 8

Nodemailer is a Node.js module for sending emails. ASTNS uses Nodemailer with Gmail SMTP (port 465, SSL) to dispatch HTML-formatted notification emails. The email template includes the student's name, the guardian's name, and a styled "View Report" button containing the token URL.

### 4.1.9 Vercel

Vercel is a cloud platform for frontend frameworks and serverless functions. ASTNS deploys:

- The React SPA as a static site served from Vercel's global CDN.
- The Express.js backend as Vercel serverless functions (via `vercel.json` routing configuration).

### 4.1.10 Git and GitHub

Git was used for version control throughout the project. The repository is hosted on GitHub with three contributors (one per team member). Feature branches were used for each major module, with pull requests and code review before merging into the main branch.

### 4.1.11 Visual Studio Code

VS Code was the primary IDE used by the team. Extensions used include: ESLint, Prettier, Tailwind CSS IntelliSense, MongoDB for VS Code, REST Client, GitLens.

### 4.1.12 Postman

Postman was used for API testing during backend development. Collections were created for each resource group (Auth, Students, Tokens, Notifications, Report).

## 4.2 Algorithm / Pseudocode

### 4.2.1 Token Generation and Email Dispatch Algorithm

```
ALGORITHM SendNotifications(recipients, semester, expiry):
  INPUT: recipients = [{studentId, email}], semester, expiry duration
  OUTPUT: {success_count, failure_count, details}

  hours = EXPIRY_MAP[expiry]  // 24, 48, 72, or 168
  expiresAt = CURRENT_TIME + hours * 3600 * 1000

  FOR EACH recipient IN recipients DO PARALLEL:
    student = FIND_STUDENT_BY_ID(recipient.studentId)
    IF student NOT FOUND:
      RECORD failure for recipient
      CONTINUE

    token_string = CRYPTO.RANDOM_BYTES(16).TO_HEX()
    token_doc = CREATE_TOKEN({
      student: student._id,
      token: token_string,
      semester: semester,
```

```
      expiresAt: expiresAt,
      status: "Active"
    })

    guardian_url = FRONTEND_URL + "/guardian?token=" + token_string
    email_sent = SEND_EMAIL(recipient.email, student.fullName, guardi

    IF email_sent:
      RECORD success for student
    ELSE:
      RECORD failure for student

  RETURN {success_count, failure_count, details}
END ALGORITHM
```

### 4.2.2 Token Validation Algorithm

```
ALGORITHM ValidateToken(token_string):
  INPUT: token_string (32-char hex from URL query parameter)
  OUTPUT: student academic data OR error response

  // Step 1: Auto-expire stale tokens
  UPDATE ALL TOKENS WHERE status="Active" AND expiresAt < NOW()
    SET status = "Expired"

  // Step 2: Find token
  token_doc = FIND_TOKEN({token: token_string})
    WITH POPULATE student -> semesters -> subjects -> subject_details

  IF token_doc NOT FOUND:
    RETURN HTTP 404 "Invalid or unknown token"
```

```
  IF token_doc.status == "Expired":
    RETURN HTTP 410 "This token has expired"

  // Step 3: Mark as used on first access
  IF token_doc.status == "Active":
    UPDATE token_doc SET status="Used", usedAt=NOW()

  RETURN HTTP 200 WITH student_data
END ALGORITHM
```

### 4.2.3 CGPA Auto-Calculation

```
HOOK Student.pre("save"):
  IF semesters IS EMPTY:
    SET cgpa = 0
    RETURN

  total_sgpa = SUM(semester.sgpa FOR ALL semester IN semesters)
  cgpa = ROUND(total_sgpa / COUNT(semesters), 2)
  SET this.cgpa = cgpa
END HOOK
```

## 4.3 Testing Techniques

ASTNS was tested using a combination of the following techniques:

- **Unit Testing:** Individual Mongoose schema validators were tested by attempting to save documents with invalid data (e.g., marks exceeding maximum, invalid course-branch combinations, invalid email formats) and verifying that validation errors were thrown correctly.
- **API Integration Testing:** All REST API endpoints were tested using Postman. Each endpoint was tested for: correct response status codes, correct response body structure, error handling for missing/invalid inputs, and authentication enforcement on protected routes.

- **Functional Testing:** The complete user workflows (Admin login, CSV upload, notification dispatch, Guardian Portal access) were tested manually end-to-end.
- **Boundary Testing:** Edge cases such as empty CSV files, CSVs with missing columns, tokens accessed after expiry, and concurrent notification dispatch to large student batches were tested.
- **Cross-Browser Testing:** The Admin Dashboard and Guardian Portal were tested on Google Chrome (v124), Mozilla Firefox (v125), and Microsoft Edge (v124) on Windows; and on Safari on iOS.
- **Responsive Design Testing:** The Guardian Portal was tested on viewport widths of 375px (mobile), 768px (tablet), and 1440px (desktop) using Chrome DevTools.

## 4.4 Test Cases

Table 4.1: Test Cases – Authentication Module

| TC# | Test Case | Input / Steps | Expected Result |
|---|---|---|---|
| TC1 | Admin Login – Valid | Email: admin@gmail.com, Password: correct | HTTP 200, session cookie set, redirect to Dashboard |
| TC2 | Admin Login – Wrong Password | Email: admin@gmail.com, Password: wrong | HTTP 401 “Invalid credentials” |
| TC3 | Admin Login – Unknown Email | Email: unknown@gmail.com | HTTP 404 “Admin not found” |
| TC4 | Admin Logout | Click Logout button | Session deleted, cookie cleared, redirect to Login |
| TC5 | Access Protected Route Without Session | Navigate to /admin/dashboard without login | HTTP 401, redirect to Login page |

Table 4.2: Test Cases – Data Upload Module

| TC# | Test Case | Input / Steps | Expected Result |
|---|---|---|---|
| TC6 | Upload Valid CSV | Upload correctly formatted CSV | Records parsed, preview table displayed correctly |
| TC7 | Upload CSV with Missing Columns | CSV missing “guardianEmail” column | Validation error shown, no records saved |
| TC8 | Upload CSV with Invalid Marks | Student marks ¿ maxTotalMarks | Mongoose validation error returned, record rejected |
| TC9 | Upload CSV with Invalid Course | Course not in courseBranchMap | Validation error: “not a valid course” |
| TC10 | Upload Non-CSV File | Upload .txt file | Frontend rejects file, error message shown |

Table 4.3: Test Cases – Notification Module

| TC# | Test Case | Input / Steps | Expected Result |
|---|---|---|---|
| TC11 | Send Notification – Single Student | Select 1 student, click Send | 1 token created, 1 email dispatched, success shown |
| TC12 | Send Notification – Batch | Select 20 students, click Send | 20 tokens created, 20 emails dispatched in parallel |
| TC13 | Send Notification – No Students Selected | Click Send with no selection | Frontend validation: “Select at least one student” |
| TC14 | Email Content Verification | Inspect received email | Email contains student name, “View Report” button, correct token URL |

Table 4.4: Test Cases – Guardian Portal

| **TC#** | **Test Case** | **Input / Steps** | **Expected Result** |
|---|---|---|---|
| TC15 | Valid Token Access | Open URL with valid token | Guardian Portal renders student academic data |
| TC16 | Expired Token Access | Open URL with expired token | Access Denied page with expiry message |
| TC17 | Revoked Token Access | Admin revokes token; Guardian opens URL | Access Denied page with revocation message |
| TC18 | Invalid Token | Manually enter random token in URL | HTTP 404 “Invalid or unknown token” |
| TC19 | SGPA Chart Rendering | Student has 4 semester records | Line chart displays 4 data points correctly |
| TC20 | Detention Flag Display | Student has a detained subject | Detention badge displayed on that subject row |

# Chapter 5

# Results and Discussions

## 5.1 User Interface Representation

### 5.1.1 Brief Description of Various Modules

**1. Login Module:** The Login page is the entry point for administrators. It displays a clean, centred card with email and password fields and a "Sign In" button. Client-side validation ensures both fields are filled before submission. On successful authentication, the admin is redirected to the Dashboard. On failure, an error message is shown.

**2. Admin Dashboard Module:** The Dashboard displays four summary stat cards: Total Students, Detained Students, Total Notifications Sent, and Active Tokens. Below the cards, a bar chart shows notification activity over the past 30 days. A branch distribution table shows the count of students per branch.

**3. Upload Data Module:** Administrators can drag-and-drop or browse to upload CSV or Excel files. After upload, the system parses the file and displays a paginated preview table showing all detected student records. The admin selects the target semester and clicks "Submit" to save the records to the database.

**4. Student List Module:** Displays a searchable, filterable table of all students. The admin can search by student name, URN, or CRN, and filter by course or branch. Each row shows the student's name, URN, course, branch, and current CGPA, with a link to the Student Detail page.

**5. Student Detail Module:** Shows the complete academic profile of a selected student. The page displays the student's personal information (name, URN, CRN, course, branch, admission

year), an SGPA trend line chart, and an accordion of all semesters. Each semester accordion expands to show a table of subject marks (internal, external, total, max marks, status).

**6. Send Notification Module:** The left panel displays all students with checkboxes for selection. Search and filter controls help narrow the list. The right panel provides configuration: semester selection, token expiry, and a "Send Notifications" button. A progress indicator shows dispatch status for each student.

**7. Token Management Module:** Displays all generated tokens in a table with columns: student name, URN, semester, status (Active / Used / Expired), created date, expiry date, and a Revoke button for active tokens. Administrators can filter by status.

**8. Guardian Dashboard Module:** Accessed via the token URL. Shows the student's profile, SGPA line chart, attendance ring, and expandable semester panels with subject-wise marks. Detention alerts are shown as a prominent red banner if any detained subjects exist.

**9. Access Denied Module:** Shown when an invalid, expired, or revoked token is used. Displays an appropriate message explaining why access was denied.

## 5.2 Snapshots of System with Details

### 5.2.1 Login Page

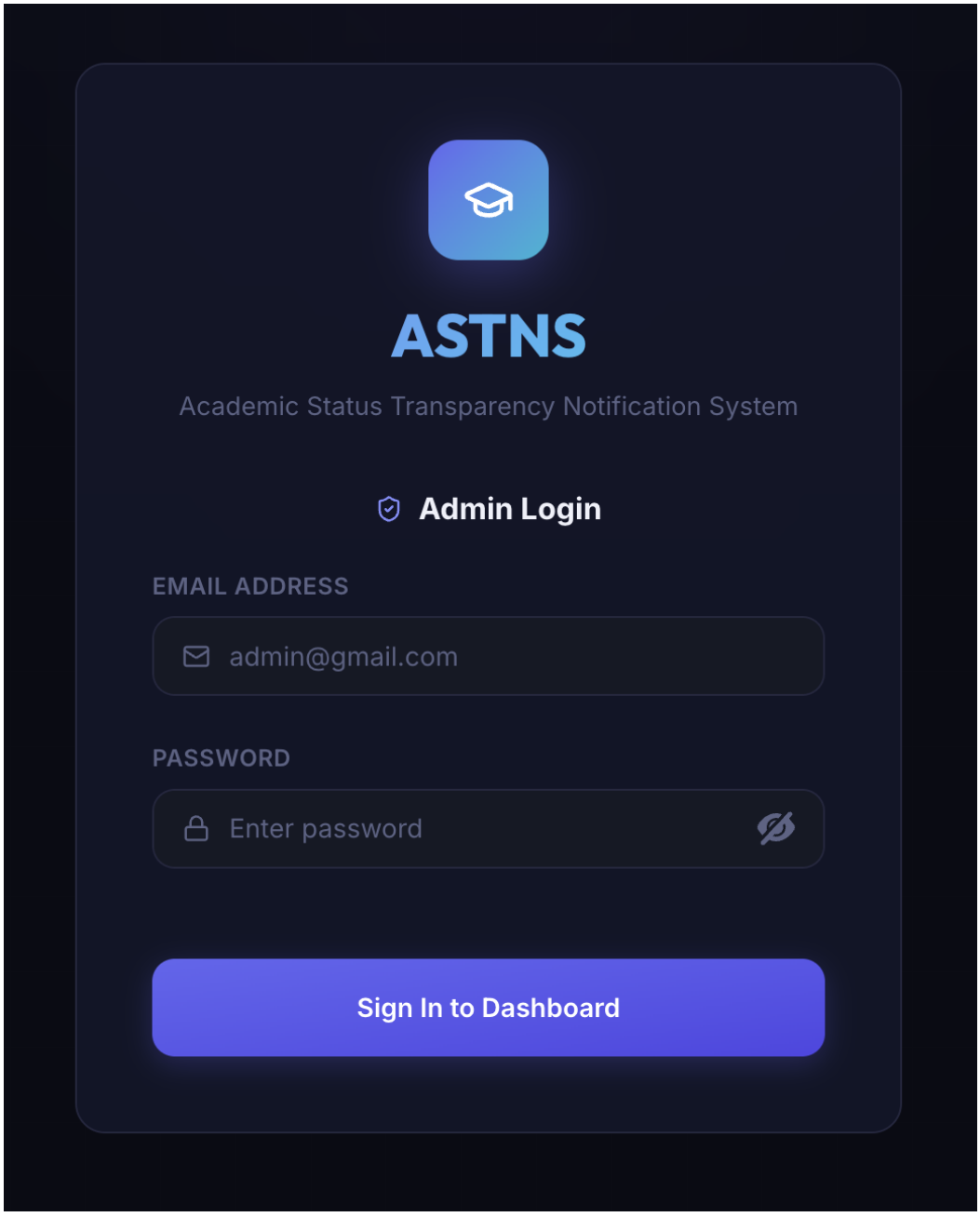


Figure 5.1: Login Page – Admin Authentication Interface

The Login page features a dark-themed card centred on the viewport. The form contains email and password inputs with client-side validation. On successful authentication, a session cookie is set and the admin is redirected to the Dashboard.

### 5.2.2 Admin Dashboard


Figure 5.2: Admin Dashboard – Overview with Stats and Charts

The Dashboard presents four metric cards at the top: Total Students, Detained Count, Notifications Sent, and Active Tokens. A bar chart below shows notification activity. The sidebar remains visible on all Admin pages for easy navigation.

### 5.2.3 Upload Data Page

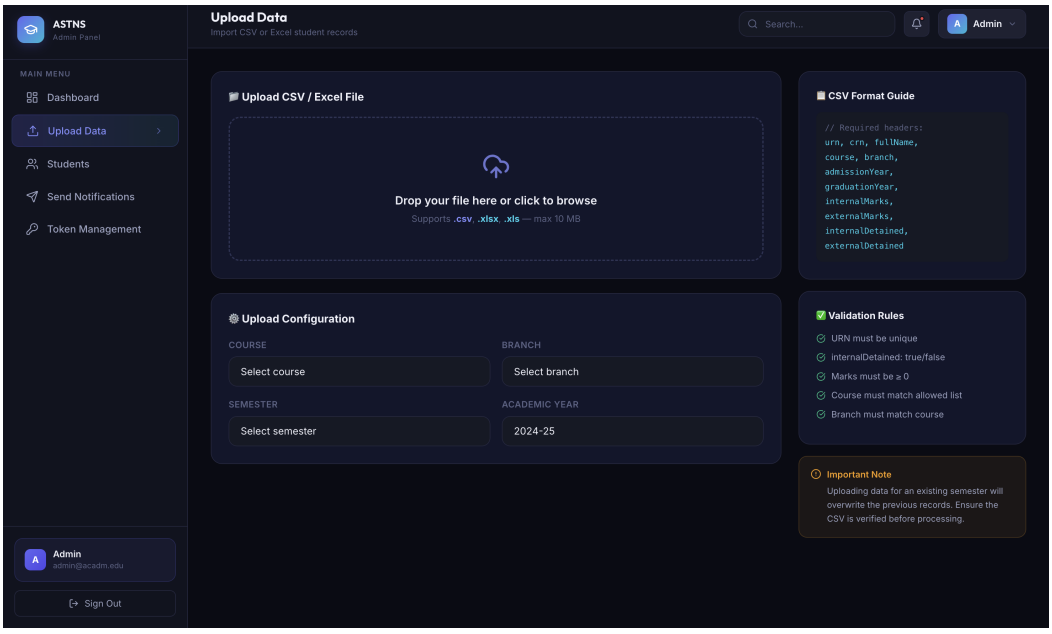


Figure 5.3: Upload Data Page – CSV Bulk Upload Interface

The Upload Data page features a drag-and-drop zone. After file selection, the system parses the file and renders a preview table. The admin selects the semester and confirms submission.

### 5.2.4 Student List Page

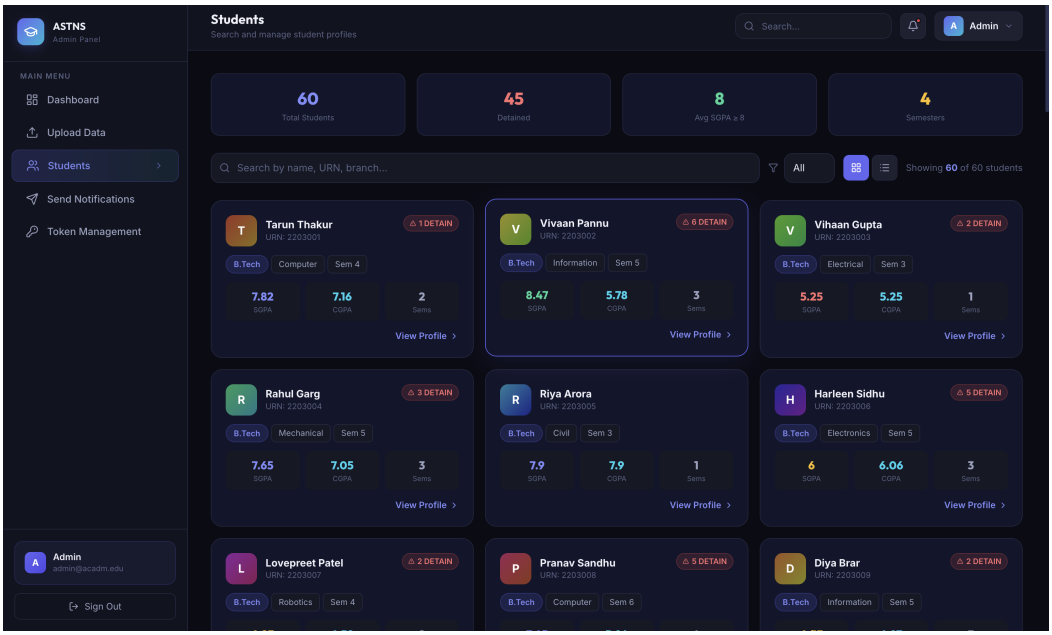


Figure 5.4: Student List Page – Searchable and Filterable Student Records

All students are displayed in a sortable table. The search bar filters by name, URN, or CRN. Dropdown filters allow selection by course and branch.

### 5.2.5 Student Detail Page

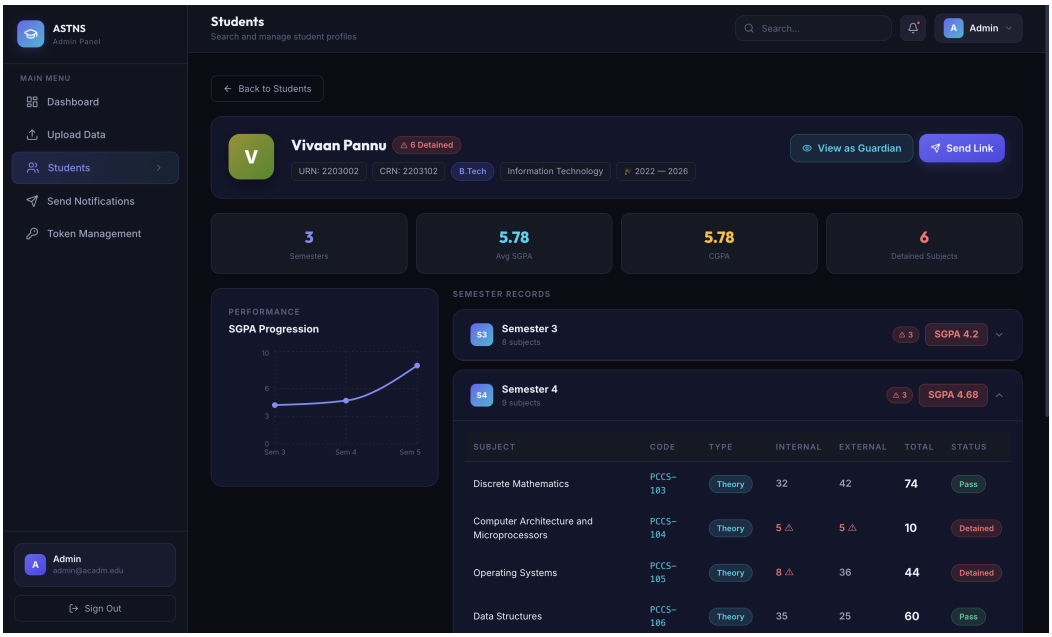


Figure 5.5: Student Detail Page – Individual Academic Profile

The Student Detail page shows the student's complete profile with an SGPA trend chart and expandable semester mark tables.

### 5.2.6 Send Notification Page

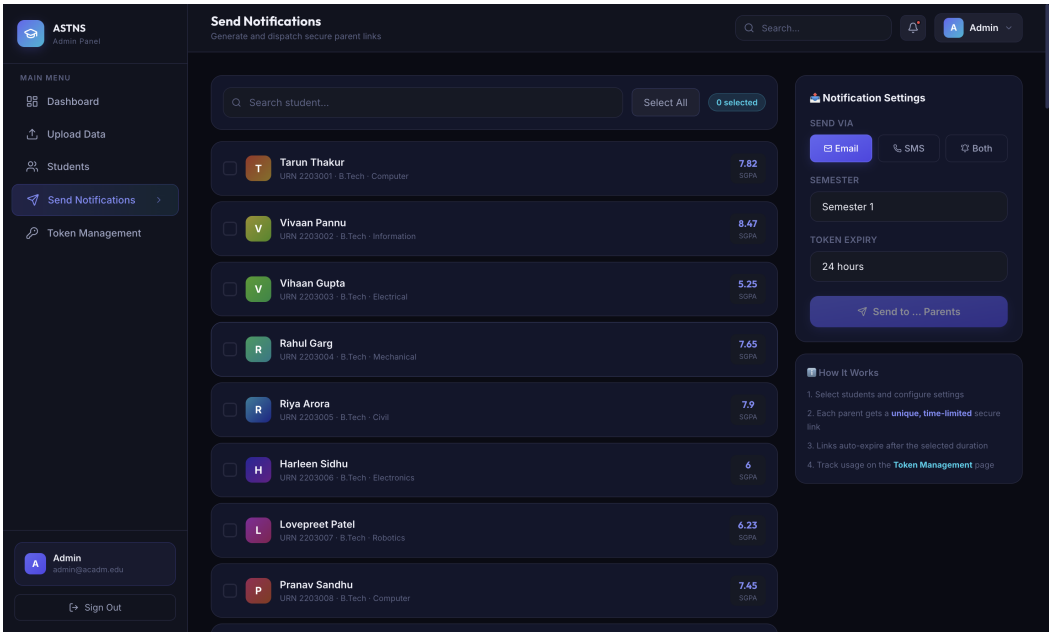


Figure 5.6: Send Notification Page – Bulk Email Dispatch Interface

Administrators select students via checkboxes and configure the notification parameters. Dispatch status is shown for each student after sending.

### 5.2.7 Token Management Page

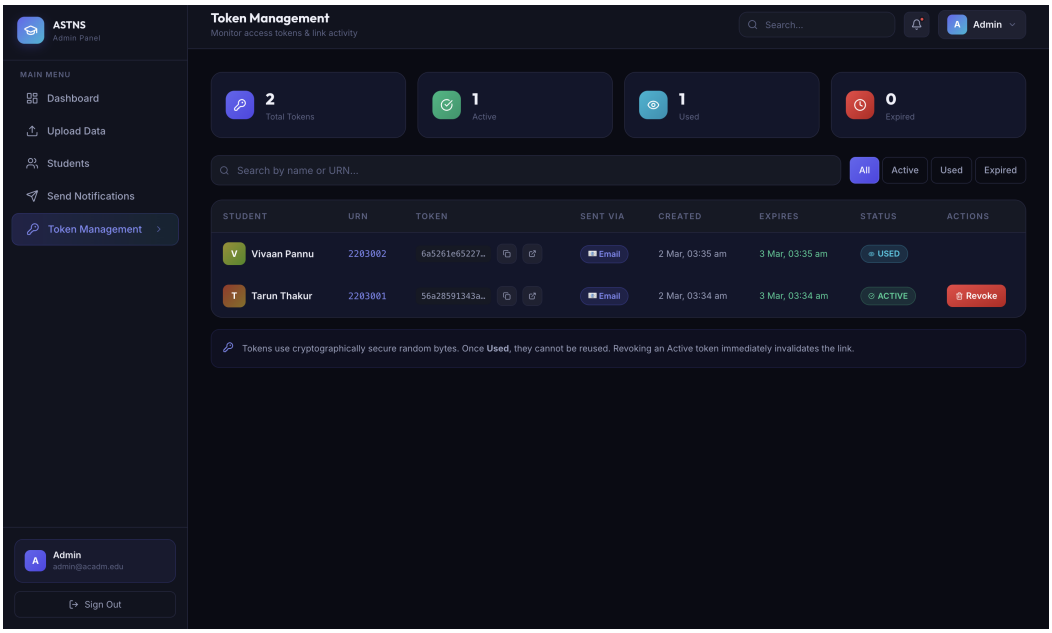


Figure 5.7: Token Management Page – Token Lifecycle Dashboard

All tokens are listed with their current status. Active tokens show a “Revoke” button. Expired tokens are greyed out.

### 5.2.8 Guardian Dashboard – Part 1


Figure 5.8: Guardian Dashboard – Student Profile and SGPA Chart

The Guardian Dashboard opens after token validation. The student's profile card and SGPA line chart are shown at the top of the page.

### 5.2.9 Guardian Dashboard – Part 2

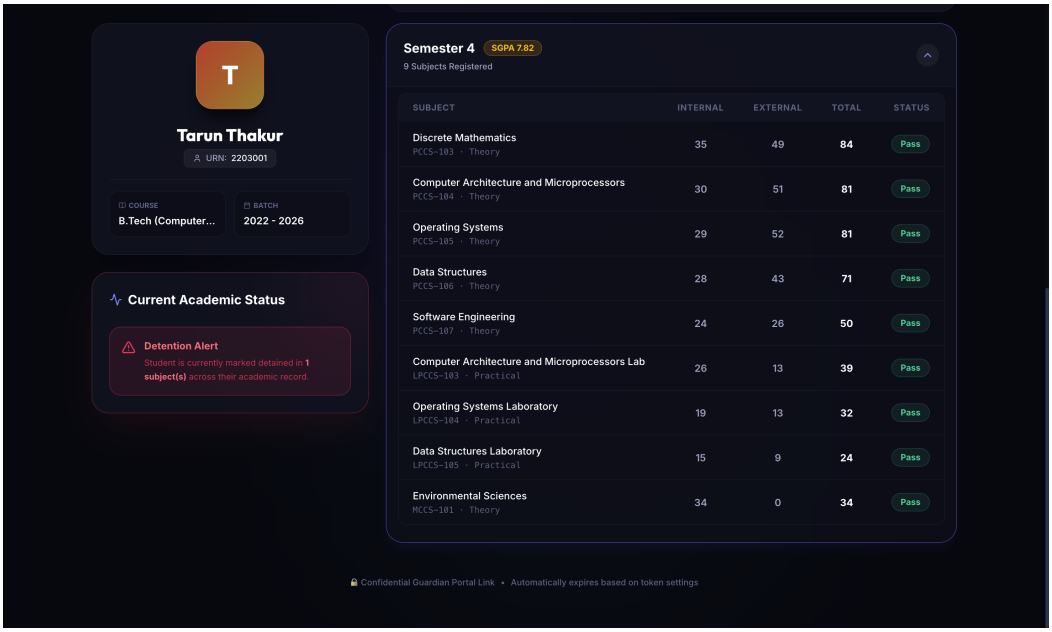


Figure 5.9: Guardian Dashboard – Semester-wise Mark Breakdown

Below the chart, expandable semester panels display subject-wise marks with detention status indicators.

### 5.2.10 Access Denied Page

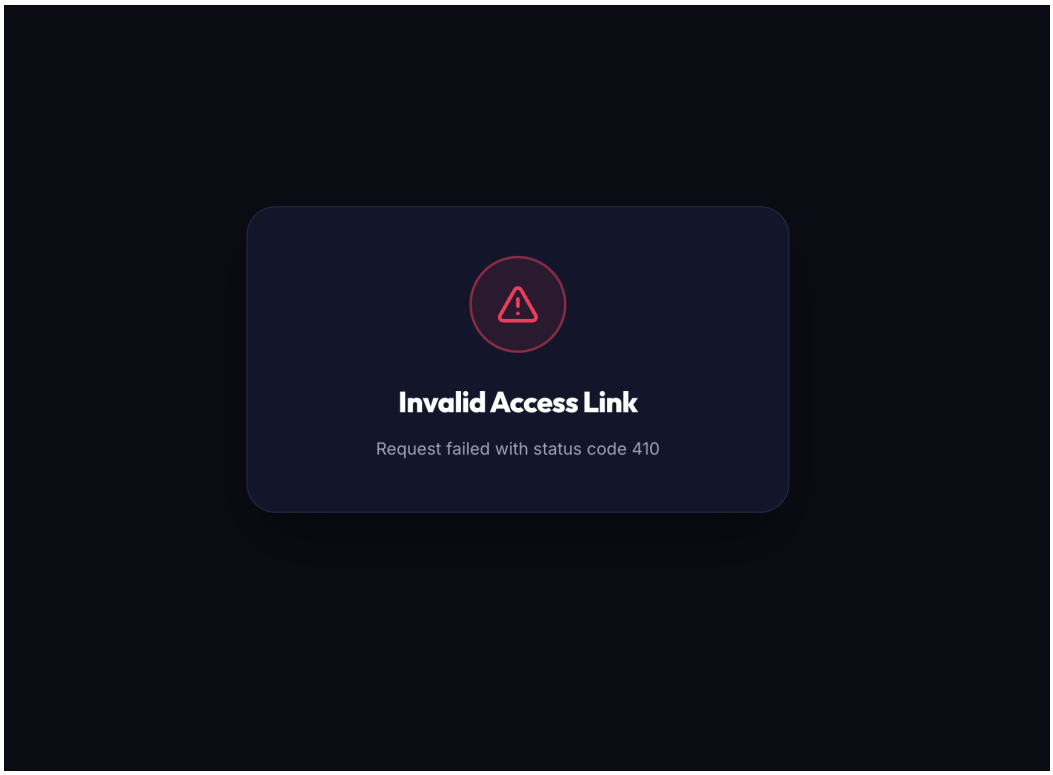


Figure 5.10: Access Denied Page – Expired or Revoked Token

When an invalid, expired, or revoked token is used, the Access Denied page is displayed with an appropriate explanation.

### 5.2.11 Email Notification

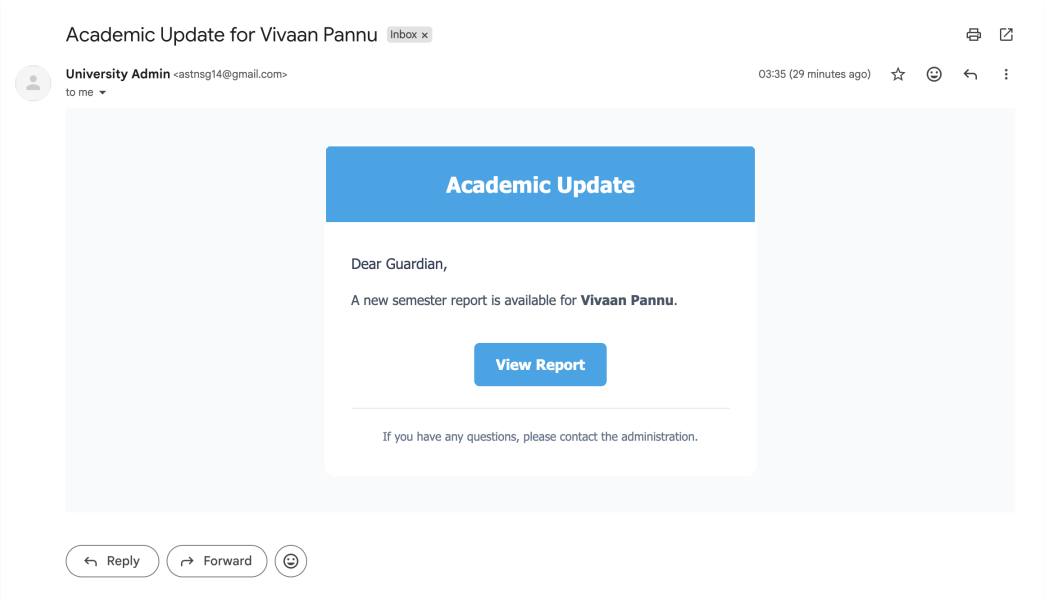


Figure 5.11: Email Notification – Guardian Notification Email Template

The HTML email dispatched to guardians contains a personalised greeting with the student's name, a brief description of the system, and a prominently styled "View Report" button linking to the Guardian Portal.

## 5.3 Backend Database Representation

### 5.3.1 MongoDB Atlas Collections

The MongoDB Atlas database for ASTNS contains the following collections:

- **admins:** Stores administrator credentials. Passwords are stored as bcrypt hashes with salt rounds of 12. Each admin document contains: `userName`, `email`, `password` (hashed), and `role`.
- **students:** The primary collection. Each student document contains nested `semesters` arrays, and each semester contains nested `subjects` arrays (each referencing a Subject document by ObjectId). The hierarchical document structure of MongoDB is ideally suited to this nested data model.

- **subjects:** A reference collection storing subject master data (title, code, type, credits, max marks, pass marks). Subject records are shared across students via ObjectId references, ensuring consistency.

- **tokens:** Stores all generated access tokens with their lifecycle metadata (status, expiry, usedAt). Automatic expiry is enforced by querying and updating expired tokens on each token-related API call.

- **sessions:** Stores admin session records linked to admin ObjectIds. Session-based authentication is used instead of JWT to support server-side session invalidation on logout.

### 5.3.2 Database Performance

MongoDB's document model allows the complete student academic profile (all semesters, all subjects) to be fetched in a single query with `populate()`, avoiding expensive multi-table JOIN operations required in relational databases. The `lean()` option is used on read-only queries to return plain JavaScript objects instead of full Mongoose documents, improving performance by approximately 25–30%.

## 5.4 Discussions

The successful implementation and deployment of ASTNS demonstrates that a modern full-stack MERN application can effectively solve a real institutional communication problem with minimal infrastructure cost.

Key findings from the testing and evaluation phase:

1. **Email Deliverability:** All test emails dispatched via Gmail SMTP were successfully delivered to recipient inboxes. No emails were flagged as spam during testing, owing to the use of an authenticated Gmail account with App Password authentication.

2. **Token Security:** The 32-character hexadecimal token space ($16^{32} = 2^{128}$ possible values) is computationally infeasible to brute-force, providing security equivalent to a 128-bit symmetric key.

3. **Batch Performance:** Notification dispatch to a batch of 50 students using `Promise.all` completed within 8–12 seconds, with all emails delivered successfully. The parallel processing approach significantly outperformed sequential dispatch.

4. **Guardian Portal Load Time:** The Guardian Portal loaded student data (including full semester and subject population) within 1.2–1.8 seconds on a standard broadband connection, meeting the 2-second performance requirement.

5. **Cross-Browser Compatibility:** The application rendered correctly and all features functioned as expected on Chrome, Firefox, Edge, and Safari. The responsive design maintained usability on screens as small as 375px (iPhone SE viewport).

6. **Data Integrity:** Mongoose schema validation successfully rejected all invalid data inputs during CSV upload testing, including out-of-range marks, invalid email formats, and invalid course-branch combinations.

# Chapter 6

# Conclusion and Future Scope

## 6.1 Conclusion

The Academic Status Transparency Notification System (ASTNS) successfully addresses the critical communication gap between engineering colleges and the parents or guardians of enrolled students. The system demonstrates that a well-designed full-stack web application, built on freely available open-source technologies, can deliver significant institutional value at near-zero operational cost.

The project achieved all six of its stated objectives:

1. A secure, session-authenticated Admin Dashboard was designed and developed, allowing institutional administrators to manage student academic data efficiently.

2. A bulk data upload module was implemented that accepts CSV and Excel files, validates records against strict schema constraints, and persists them to a cloud database.

3. An automated email notification pipeline was developed that generates cryptographically secure, time-limited access tokens and dispatches personalised HTML emails to guardians.

4. A token-authenticated Guardian Portal was created that renders a student's complete academic profile without requiring guardians to create an account.

5. A Token Management module was implemented that allows administrators to monitor and revoke access tokens.

6. The complete application was deployed on Vercel as a serverless application, ensuring high availability and automatic scaling.

The system was successfully tested across all major user workflows and use cases. All functional requirements were met. The application demonstrated strong cross-browser compatibility, responsive design for mobile devices, and acceptable performance under batch notification scenarios.

The use of MongoDB's document model proved particularly advantageous for the nested, hierarchical structure of student academic data (student $\rightarrow$ semesters $\rightarrow$ subjects). The serverless deployment model eliminated infrastructure management concerns, making the system suitable for adoption by resource-constrained educational institutions.

Most importantly, ASTNS proves the concept that password-less, token-based access mechanisms borrowed from banking and e-commerce domains can be effectively applied to educational notification systems. This approach maximises guardian engagement by removing all technical barriers to accessing academic reports.

## 6.2 Future Scope

While the current implementation of ASTNS meets all project objectives, several enhancements can significantly extend its utility and impact:

- **SMS and WhatsApp Notification:** Integrating Twilio SMS API and WhatsApp Business API would enable notification delivery via mobile messaging platforms, reaching guardians who do not actively use email. This is particularly relevant for semi-urban and rural demographics.

- **PDF Report Generation:** Implementing server-side PDF generation (using libraries such as Puppeteer or PDFKit) would allow guardians to download official-looking academic reports with the institution's letterhead, which could serve as documentation for official purposes.

- **AI-Powered Academic Risk Prediction:** Integrating a machine learning model trained on historical SGPA trends and attendance patterns could generate early warning flags for

students at risk of academic failure or detention, enabling proactive intervention by both guardians and administrators.

- **Multi-Institution Support:** Extending the system to a multi-tenant architecture would allow multiple colleges within the IKGPTU affiliation to use a single deployment with isolated data namespaces.

- **Two-Way Communication:** Adding a messaging feature allowing guardians to raise academic queries or request parent-teacher meetings directly through the Guardian Portal would transform ASTNS from a one-way notification system into a comprehensive academic communication platform.

- **Automated Scheduled Notifications:** Implementing a CRON job-based notification scheduler (using node-cron) would allow the system to automatically dispatch notifications at configurable intervals (e.g., monthly) without requiring manual administrator action.

- **Localisation / Multi-language Support:** Adding support for regional languages (Hindi, Punjabi) in the Guardian Portal would make academic reports accessible to a wider demographic of parents who are more comfortable in their native languages.

- **Analytics Dashboard:** A comprehensive analytics module showing trends in student performance across batches, branches, and semesters would provide institutional leadership with data-driven insights for academic policy decisions.

# References / Bibliography


1. C. I. P. Benablo, E. T. Sart, J. M. M. Bringula, and R. Bringula, "An Application for Monitoring and Inquiring of Students' Academic Record with Automatic Notification for the Parents," *International Journal of Cloud Computing and Services Science*, vol. 2, no. 1, pp. 59–66, 2014.

2. Meta Platforms, Inc., "React – A JavaScript library for building user interfaces," 2024. [Online]. Available: `https://react.dev`

3. OpenJS Foundation, "Express.js v5 Documentation," 2024. [Online]. Available: `https://expressjs.com`

4. MongoDB Inc., "MongoDB Atlas Documentation," 2024. [Online]. Available: `https://www.mongodb.com/docs/atlas`

5. Mongoose Contributors, "Mongoose ODM v9 Documentation," 2024. [Online]. Available: `https://mongoosejs.com/docs`

6. Nodemailer Contributors, "Nodemailer Documentation," 2024. [Online]. Available: `https://nodemailer.com`

7. Tailwind Labs, "Tailwind CSS v4 Documentation," 2024. [Online]. Available: `https://tailwindcss.com/docs`

8. Recharts Group, "Recharts – Redefined chart library built with React and D3," 2024. [Online]. Available: `https://recharts.org`

9. Vercel Inc., "Vercel Documentation – Serverless Functions," 2024. [Online]. Available: `https://vercel.com/docs`

10. React Hook Form Contributors, “React Hook Form Documentation,” 2024. [Online]. Available: https://react-hook-form.com

11. Node.js Foundation, “Node.js v20 LTS Documentation,” 2024. [Online]. Available: https://nodejs.org/en/docs

12. Vite Contributors, “Vite Documentation,” 2024. [Online]. Available: https://vitejs.dev

13. I.K. Gujral Punjab Technical University, “University Official Portal,” 2024. [Online]. Available: https://www.ptu.ac.in

14. GNDEC Minor Project Synopsis – Group G14, Department of CSE, Guru Nanak Dev Engineering College, Ludhiana, 2025.

# Appendix A

# Project Setup Guide

## A.1 Prerequisites

- Node.js v20 LTS or higher installed.
- A MongoDB Atlas account with an M0 cluster created.
- A Gmail account with App Password generated (2FA must be enabled).
- Git installed.
- Vercel CLI installed globally (`npm install -g vercel`).

## A.2 Backend Setup

```
# Clone repository
git clone https://github.com/KingArsh05/minorProjectG14.git
cd minorProjectG14/server

# Install dependencies
npm install

# Create .env file with:
MONGODB_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/astns
COOKIE_SECRET=your_cookie_secret_key
```

```
FRONTEND_URL=http://localhost:5173
SMTP_USER=yourgmail@gmail.com
SMTP_PASS=your_app_password
NODE_ENV=development

# Start development server
npm run dev
```

## A.3 Frontend Setup

```
cd minorProjectG14/client

# Install dependencies
npm install

# Create .env file with:
VITE_API_URL=http://localhost:3000

# Start development server
npm run dev
```

## A.4 Deployment to Vercel

```
# Login to Vercel
vercel login

# Deploy from project root
vercel

# Set environment variables in Vercel dashboard:
# MONGODB_URI, COOKIE_SECRET, FRONTEND_URL, SMTP_USER, SMTP_PASS
```